

TextWriter and TextReader in C# with Examples

In this article, I am going to discuss **TextWriter and TextReader in C#** with Examples. Please read our previous article where we discussed **File Class in C#** with Examples. At the end of this article, you will understand what TextWriter and TextReader are in C# and when and how to use TextWriter and TextReader in C# with Examples.

TextWriter and TextReader Classes in C#:

The TextReader and TextWriter classes in C# are another way to read and write files respectively, even though these are not stream classes. The TextReader and TextWriter are base classes. The StreamReader and StringReader derive from the abstract type TextReader. Similarly, the StreamWriter and StringWriter derive from the abstract type TextWriter.

What is TextWriter Class in C#?

The TextWriter class in C# represents a writer that can write sequential series of characters. We can use this TextWriter class to write text in a file. It is an abstract base class of StreamWriter and StringWriter, which write characters to streams and strings respectively. It is used to write text or sequential series of characters into files. It is found in the **System.IO** namespace. If you go to the definition of TextWriter Class, you will see that it is an abstract class as shown in the below image.

```
namespace System.IO
{
    ...public abstract class TextWriter : MarshalByRefObject, IDisposable
    {
        ...public static readonly TextWriter Null;
        ...protected char[] CoreNewLine;

        ...protected TextWriter();
        ...protected TextWriter(IFormatProvider formatProvider);

        ...public virtual IFormatProvider FormatProvider { get; }
        ...public abstract Encoding Encoding { get; }
        ...public virtual string NewLine { get; set; }
```

Methods of TextWriter class in C#:

1. **Synchronized(TextWriter)**: It is used to Create a thread-safe wrapper around the specified TextWriter.
2. **Close()**: It Closes the current writer and releases any system resources associated with the writer.
3. **Dispose()**: It releases all resources used by the System.IO.TextWriter object.
4. **Flush()**: It Clears all buffers for the current writer and causes any buffered data to be written to the underlying device.
5. **Write(Char)**: It is used to write a character to the text stream.
6. **Write(String)**: It is used to write the string to the text stream.

7. **WriteAsync(Char)**: It is used to write the character to the text stream asynchronously.
8. **WriteLine()**: It is used to write a line terminator to the text stream.
9. **WriteLineAsync(String)**: It is used to write the string to the text stream asynchronously followed by a line terminator.

Note: There are many overloaded versions of Write and WriteAsync methods available in TextWriter class.

Points to Remember:

1. TextWriter is an abstract class belonging to the System.IO namespace.
2. It is used to write a sequential series of characters into a file.
3. It is the base class of StreamWriter and StringWriter which is used to write characters to streams and strings respectively.
4. By default, it is not thread-safe.
5. As it is an abstract class, its object cannot be created. Any class implementing TextWriter must minimally implement its Write(Char) method to create its useful instance.

How TextWriter work in C#?

To work with TextWriter in C#, first, we need to import the System.IO namespace. We cannot directly create an instance of TextWriter using the new keyword as it is an abstract class. So, to create the instance we need to use the CreateText() method of the File class as follows:

```
TextWriter textWriter = File.CreateText(filePath);
```

The File.CreateText method takes the path of the file to be opened for writing. It creates or opens a file for writing UTF-8 encoded text. If the file already exists, then its content will be overwritten. The File.CreateText(filePath) method returns an instance of StreamWriter class, which is one of the derived classes of the TextWriter abstract class. Now, with the help of this instance, we can call the methods of TextWriter class to write text into a file.

Like StreamWriter there are also other classes that are derived from the TextWriter class and provide the implementation for the members of TextWriter. The following is the list of derived classes with the help of which we can work with TextWriter:

1. **IndentedTextWriter**: It is used to insert a tab string and to track the current indentation level.
2. **StreamWriter**: It is used to write characters to a stream in a particular encoding.
3. **StringWriter**: It is used to write information to a string. The information is stored in an underlying StringBuilder.
4. **HttpWriter**: It provides an object of TextWriter class that can be accessed through the intrinsic HttpResponse object.
5. **HtmlTextWriter**: It is used to write markup characters and text to an ASP.NET server control output stream.

Example to Understand TextWriter Class in C#:

In the below example, first, we create one string variable which holds the file path. Then we create an instance of TextWriter class and to create an instance we call the File.CreateText method and provide the file path where we want to create the file. Then using the WriteLine method we write some data into the file.

```

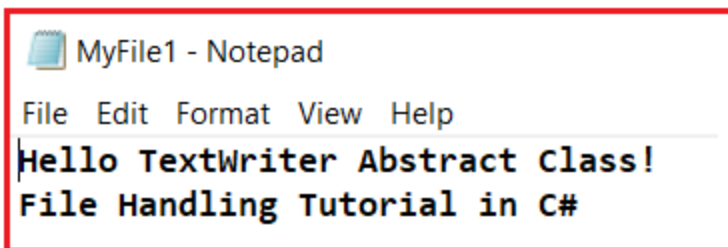
using System;
using System.IO;
namespace FileHandlinDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            string filePath = @"D:\MyFile1.txt";
            using (TextWriter textWriter = File.CreateText(filePath))
            {
                textWriter.WriteLine("Hello TextWriter Abstract Class!");
                textWriter.WriteLine("File Handling Tutorial in C#");
            }
            Console.WriteLine("Write Successful");
            Console.ReadKey();
        }
    }
}

```

When you run the above code, you will get the following output.

Write Successful

Now, you can verify that the **MyFile1.txt** file should be created inside the D drive with the following data.



We can also asynchronously write characters to stream by using WriteAsync(Char) method which is shown in the below example.

```

using System;
using System.IO;
namespace FileHandlingDemo
{
    public class Program
    {
        public static void Main(string[] args)
        {
            WriteCharAsync();
            Console.ReadKey();
        }
        public static async void WriteCharAsync()
        {

```

```

        string filePath = @"D:\MyFile2.txt";
        using (TextWriter textWriter = File.CreateText(filePath))
        {
            await textWriter.WriteLineAsync("Hello TextWriter Abstract Class");
            await textWriter.WriteLineAsync("File Handling Tutorial in C#");
        }
        Console.WriteLine("Async Write Successful");
    }
}

```

Note: The TextWriter class in C# is used to write text or sequential series of characters to a file. A class derived from the TextWriter class needs to provide implementation to the members of the TextWriter class.

What is TextReader class in C#?

The TextReader class in C# represents a reader that is used to read text or sequential series of characters from a text file. TextReader class belongs to the System.IO namespace. It is an abstract class which means you cannot instantiate it. It is an abstract base class of StreamReader and StringReader which are used to read characters from stream and string respectively. If you go to the definition of TextReader Class, you will see that it is an abstract class as shown in the below image.

```

namespace System.IO
{
    ...public abstract class TextReader : MarshalByRefObject, IDisposable
    {
        ...public static readonly TextReader Null;

        ...protected TextReader();

        ...public static TextReader Synchronized(TextReader reader);
        ...public virtual void Close();
        ...public void Dispose();
        ...public virtual int Peek();
        ...public virtual int Read();
        ...public virtual int Read(char[] buffer, int index, int count);
        ...public virtual Task<int> ReadAsync(char[] buffer, int index, int count);
        ...public virtual int ReadBlock(char[] buffer, int index, int count);
        ...public virtual Task<int> ReadBlockAsync(char[] buffer, int index, int count);
        ...public virtual string ReadLine();
        ...public virtual Task<string> ReadLineAsync();
        ...public virtual string ReadToEnd();
        ...public virtual Task<string> ReadToEndAsync();
        ...protected virtual void Dispose(bool disposing);
    }
}

```

Methods of TextReader Class in C#:

1. **Synchronized():** It is used to create a thread-safe wrapper around the specified TextReader.

2. **Close():** It is used to close the TextReader and release any system resources associated with the TextReader.
3. **Dispose():** It is used to release all resources used by the TextReader object.
4. **Peek():** It is used to read the next character without changing the state of the reader or the character source. Returns the next available character without actually reading it from the reader. It returns an integer representing the next character to be read, or -1 if no more characters are available or the reader does not support seeking.
5. **Read():** It is used to read the next character from the text reader and advances the character's position by one character. It returns the next character from the text reader, or -1 if no more characters are available. The default implementation returns -1.
6. **ReadLine():** It is used to read a line of characters from the text reader and returns the data as a string. It returns the next line from the reader, or null if all characters have been read.
7. **ReadToEnd():** It is used to read all characters from the current position to the end of the text reader and returns them as one string. That means it reruns a string that contains all characters from the current position to the end of the text reader.

How Does TextReader Work in C#?

We cannot create an instance of TextReader class in C# because it is an abstract class. TextReader is not threaded safe by default. The class deriving the TextReader class needs to minimally implement Peek() and Read() methods to make a useful instance of TextReader.

As TextReader is an abstract class, we cannot create its instance directly using the new keyword but we can call the OpenText() method of the File class to achieve the same. The OpenText() method takes the location of the file as input and then it opens an existing UTF-8 encoded text file at the same location for reading. The Syntax of creating TextReader is shown below:

```
TextReader textReader = File.OpenText(filePath);
```

The File.OpenText() method returns an object of the StreamReader class which is the derived class of TextReader and thus helps in creating a useful instance of TextReader class. This instance can be used to call the methods of TextReader class to read content from the file. We can also create a TextReader instance with the help of using block as follows:

```
using(TextReader textReader = File.OpenText(filePath))  
{  
//Code  
}
```

The advantage of working with using block is that it releases the memory acquired by the textReader as soon as we move from the using block. We can work with TextReader with the help of its two derived classes i.e. StreamReader and StringReader.

1. **StreamReader:** It is used to read characters from a byte stream in a particular encoding.
2. **StringReader:** It is used to read text from a string.

Example to Understand TextReader Class in C#:

In the below example, we will open the D:\MyFile1.txt file (which we just created in our previous example) using the TextReader class, and then read the file and print the data on the console.

```

using System;
using System.IO;
namespace FileHandlinDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            string filePath = @"D:\MyFile1.txt";
            //Read One Line
            using (TextReader textReader = File.OpenText(filePath))
            {
                Console.WriteLine(textReader.ReadLine());
            }
            //Read 4 Characters
            using (TextReader textReader = File.OpenText(filePath))
            {
                char[] ch = new char[4];
                textReader.ReadBlock(ch, 0, 4);
                Console.WriteLine(ch);
            }
            //Read full file
            using (TextReader textReader = File.OpenText(filePath))
            {
                Console.WriteLine(textReader.ReadToEnd());
            }
            Console.ReadKey();
        }
    }
}

```

the

Output:

```

Hello TextWriter Abstract Class!
Hell
Hello TextWriter Abstract Class!
File Handling Tutorial in C#

```

In the next article, I am going to discuss [BinaryWriter and BinaryReader in C#](#) with Examples. Here, in this article, I try to explain **TextWriter and TextReader Classes in C#** with Examples. I hope you enjoy this TextWriter and TextReader in C# with Examples article. I would like to have your feedback. Please post your feedback, question, or comments about this article.